LAB 3: Introduction to Python – III

Artificial Intelligence – Lab Instructions | University of Central Punjab

3.1 Strings and String Methods

Strings (Lab 1, section 1.1.3.2) come with many built-in methods. Strings are immutable, so a method never changes the original string; it returns a new one.

```
text = "  Artificial Intelligence  "
print(text.strip())                 # 'Artificial Intelligence'
print(text.upper())                 # '  ARTIFICIAL INTELLIGENCE  '
print(text.lower())                 # '  artificial intelligence  '
print(text.replace("Artificial", "Natural"))
print(text.strip().split(" "))      # ['Artificial', 'Intelligence']
print("-".join(["a", "b", "c"]))    # a-b-c
print("python".startswith("py"))    # True
print("banana".count("a"))          # 3
print("banana".find("n"))           # 2  (index of first match, -1 if none)
print(len("python"))                # 6
```

Method	What it does
`strip()`	Removes spaces from both ends
`upper()` / `lower()`	Changes the case
`replace(old, new)`	Replaces every `old` with `new`
`split(sep)`	Splits a string into a list
`join(list)`	Joins a list of strings into one string
`find(sub)`	Index of the first match, or -1
`count(sub)`	Number of times `sub` appears

An **f-string** puts values inside a string. Put `f` before the quote and write variables inside `{}`:

```
name = "Ali"
marks = 87.456
print(f"{name} scored {marks:.1f}")   # Ali scored 87.5
```

3.2 The while Loop, break and continue

A `while` loop repeats as long as its condition is true.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

`break` stops a loop immediately. `continue` skips to the next iteration.

```
for i in range(1, 11):
    if i % 2 == 0:
        continue      # skip even numbers
    if i > 7:
        break         # stop the loop
    print(i)          # prints 1 3 5 7
```

3.3 User Input and Type Conversion

`input()` always returns a string. Convert it with `int()` or `float()` before doing arithmetic.

```
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)

price = float(input("Enter a price: "))
print("With 10% tax:", price * 1.10)
```

3.4 Handling Errors with try and except

If the user types text where a number is expected, `int()` crashes the program. A `try` / `except` block catches the error instead.

```
try:
    number = int(input("Enter a number: "))
    print(10 / number)
except ValueError:
    print("That was not a number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

3.5 List Comprehensions

A list comprehension builds a list in one line.

```
squares = [x ** 2 for x in range(1, 6)]            # [1, 4, 9, 16, 25]
evens = [x for x in range(1, 11) if x % 2 == 0]   # [2, 4, 6, 8, 10]
lengths = [len(w) for w in ["cat", "window"]]      # [3, 6]
```

The first line does the same as:

```
squares = []
for x in range(1, 6):
    squares.append(x ** 2)
```

3.6 Modules

A module is a file of ready-made code. Use `import` to load it.

```
import math
import random

print(math.sqrt(16))          # 4.0
print(math.pi)                # 3.141592653589793
print(math.factorial(5))      # 120

print(random.randint(1, 10))  # a random integer from 1 to 10
colors = ["red", "green", "blue"]
print(random.choice(colors))  # a random item from the list
```

3.7 Reading and Writing Files

Lab 1 (section 1.1.4) showed the `with` statement. The second argument of `open` is the mode:

Mode	Meaning
`'r'`	Read (default)
`'w'`	Write (erases the old contents)
`'a'`	Append (adds to the end)

```
with open('students.txt', 'w') as f:
    f.write("Ali\n")
    f.write("Sara\n")

with open('students.txt', 'a') as f:
    f.write("Umar\n")

with open('students.txt', 'r') as f:
    for line in f:
        print(line.strip())
```

3.8 Lab Tasks

Exercise 3.1

What does each expression print? Write your guess first, then run it to check.

```
s = "Hello World"
print(s.upper())
print(s.lower())
print(s[0:5])
print(s[-1])
print(s.replace("World", "Python"))
print(s.split(" "))
print(len(s))
print(s.count("l"))
print(s.find("W"))
```

Exercise 3.2

Write a function `count_vowels(text)` that returns the number of vowels (a, e, i, o, u) in a string. For example, `count_vowels("Artificial")` returns 5.

Exercise 3.3

Write a function `is_palindrome(text)` that returns `True` if a word reads the same forwards and backwards (e.g. `"madam"`), otherwise `False`. Ignore upper/lower case.

Exercise 3.4

Write a program that keeps asking the user for numbers using a `while` loop until the user types `done`. Then print the sum and the average of the numbers entered. Use `try` / `except` so that invalid input prints a message instead of crashing.

Exercise 3.5

Guessing game. Use `random.randint(1, 20)` to pick a secret number. Ask the user to guess in a `while` loop and print "Too high" or "Too low" after each guess. When the guess is right, stop the loop with `break` and print how many guesses it took.

Exercise 3.6

Using list comprehensions, create:

1. A list of the squares of the numbers from 1 to 20.
2. A list of the multiples of 3 between 1 and 50.
3. A list of the lengths of the words in `["python", "is", "easy", "to", "learn"]`.

Exercise 3.7

1. Write five student names to a file `names.txt`, one per line.
2. Read the file and print each name with a number, e.g. `1. Ali`.
3. Append one more name to the file.
4. Read the file again and print how many names it contains.

Exercise 3.8

Read a sentence from the user. Use `split()` and a dictionary (Lab 2, section 2.6) to count how many times each word appears, then print the result. For example, `"the cat saw the dog"` gives `the: 2`, `cat: 1`, `saw: 1`, `dog: 1`.

Exercise 3.9

Read a full name from the user (e.g. `ali raza`). Remove the extra spaces, then print it with the first letter of each word capitalised (use `strip()` and `title()`), its length, and the name in upper case.

Exercise 3.10

Read a sentence from the user and print:

1. The number of words.
2. The number of characters, with and without spaces.
3. The sentence with the words in reverse order (use `split()`, `reverse()` and `join()`).
4. The longest word in the sentence.

Exercise 3.11

Write a function `reverse_string(text)` that returns a string reversed, first using a `for` loop and then using slicing (`text[::-1]`). Check that both give the same result for `"python"`.

Exercise 3.12

Write a function `count_characters(text)` that returns a dictionary with the number of times each character appears. For example, `count_characters("banana")` returns `{'b': 1, 'a': 3, 'n': 2}`.

Exercise 3.13

Temperature converter. Show a menu with two options: Celsius to Fahrenheit (`F = C * 9/5 + 32`) and Fahrenheit to Celsius. Keep showing the menu in a `while` loop until the user chooses Exit. Use `try` / `except` for invalid numbers and print results with one decimal place using an f-string.

Exercise 3.14

Write a program that asks the user for a positive integer `n` and prints:

1. All the even numbers from 1 to `n`.
2. The sum of all the numbers from 1 to `n`.
3. The factorial of `n` using a `while` loop (compare it with `math.factorial(n)`).

Exercise 3.15

Write a function `is_prime(n)` that returns `True` if `n` is prime. Then use a list comprehension to build a list of all prime numbers between 1 and 100 and print it.

Exercise 3.16

Print the multiplication tables of 2 to 5 side by side using nested `for` loops, formatted like this:

```
2 x 1 = 2    3 x 1 = 3    4 x 1 = 4    5 x 1 = 5
2 x 2 = 4    3 x 2 = 6    4 x 2 = 8    5 x 2 = 10
```

Continue up to 10.

Exercise 3.17

Write a program that generates a list of 10 random integers between 1 and 100 using `random.randint()`. Then print:

1. The list, the largest and the smallest value (use `max()` and `min()`).
2. The sum and the average.
3. The list sorted in ascending and descending order.
4. Only the numbers greater than the average (use a list comprehension).

Exercise 3.18

Student marks file.

1. Create a file `marks.txt` where each line has a name and a mark separated by a comma, e.g. `Ali,85`. Enter at least five students.
2. Read the file and store the data in a dictionary (`name: mark`). Use `split(",")` and `int()`.
3. Print each student's name and mark, the class average, and the name of the student with the highest mark.
4. Write the names of the students who scored below the average to a new file `below_average.txt`.

Exercise 3.19

Password checker. Write a function `check_password(password)` that returns `True` only if the password has at least 8 characters, contains at least one digit, and contains at least one upper case letter. (Hint: use `isdigit()` and `isupper()` inside a loop.) Keep asking the user for a password until a valid one is entered.

Exercise 3.20

Mini contact book. Use a dictionary to store contacts (`name: phone number`) and show a menu in a `while` loop with these options: add a contact, search for a contact, delete a contact, show all contacts, and exit. Save the contacts to a file `contacts.txt` when the user exits, and load them from the file when the program starts.